

MANUAL TECNICO DICRI

Elaborado por: Angel Juarez

Fecha: 21/11/2025

Versión 1.0

CONTENIDO

I.	Introducción	3
1.	Objetivo.....	3
II.	Arquitectura	3
III.	Requerimientos Tecnológicos.....	5
	Requerimientos para Frontend	5
	Requerimientos para Backend.....	5
IV.	Seguridad y Comunicaciones.....	6
V.	Conexión y configuración de la base de datos.....	6

I. Introducción

1. Objetivo

Proporcionar la información técnica necesaria para la instalación, configuración, despliegue y mantenimiento del Sistema DICRI, abarcando los componentes frontend, backend y base de datos.

II. Arquitectura

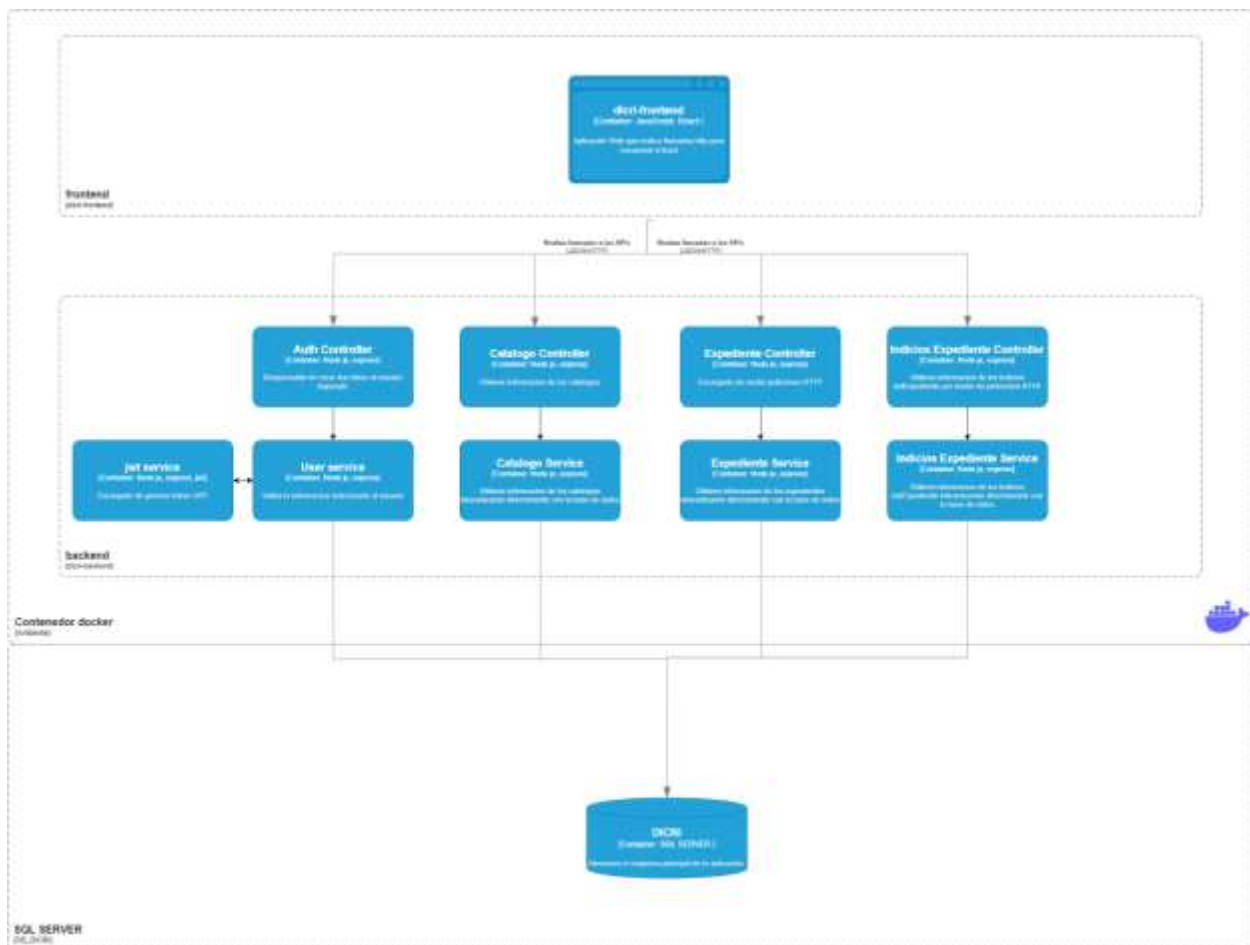
La solución implementa una arquitectura web contenerizada que separa claramente las responsabilidades entre el cliente y el servidor. En la capa de presentación (Frontend), se encuentra el componente **"dicri-frontend"**, desarrollado en **ReactJS**, el cual funciona como la interfaz de usuario encargada de realizar peticiones HTTP enviando y recibiendo datos en formato JSON hacia el servidor.

El Backend, denominado **"dicri-backend"**, se ejecuta dentro de un contenedor Docker utilizando **Node.js** y **Express**. Este sigue un patrón de diseño en capas dividido en **Controladores** (encargados de recibir las peticiones HTTP) y **Servicios** (encargados de la lógica de negocio y acceso a datos). El backend se organiza en cuatro módulos funcionales principales:

1. **Módulo de Autenticación:** Compuesto por el Auth Controller y el User service, los cuales interactúan con un componente específico llamado jwt service dedicado exclusivamente a la generación de tokens JWT para la seguridad.
2. **Módulo de Catálogos:** Gestionado por el Catalogo Controller y Catalogo Service.

3. **Módulo de Expedientes:** Gestionado por el Expediente Controller y Expediente Service.
4. **Módulo de Indicios:** Gestionado por el Indicios Expediente Controller y su respectivo servicio.

la persistencia de datos se maneja en un contenedor independiente de **SQL Server** (etiquetado como "DICRI"), el cual almacena el esquema principal de la aplicación. Los servicios de Catálogo, Expediente e Indicios se conectan directamente a esta base de datos para obtener y persistir la información, cerrando el flujo de la arquitectura.



III.Requerimientos Tecnológicos

Requerimientos para Frontend

Componente	Tecnologia
Lenguajes	HTML, Javascript
Librerías	React JS
Herramientas de diseño	Tailwind CSS

Requerimientos para Backend

Componente	Tecnologia
Lenguajes	Javascript
Librerías	Express,
Entorno de Ejecución	Node Js

IV. Seguridad y Comunicaciones

Todas las comunicaciones se realizan mediante protocolo HTTPS, garantizando cifrado extremo a extremo. Se implementan políticas de CORS en el backend para permitir el acceso controlado desde dominios autorizados. Los tokens JWT caducan automáticamente para evitar accesos prolongados. Los roles dentro del sistema restringen la visibilidad de funcionalidades. Los Registros en bitácora permiten monitorear y auditar acciones críticas del sistema.

V. Conexión y configuración de la base de datos

El sistema utiliza Node.js con Express como framework para manejar las rutas y controladores de la aplicación. La conexión a la base de datos SQL Server se realiza mediante el paquete mssql, utilizando las credenciales y parámetros definidos en variables de entorno:

DB_USER=sa

DB_PASSWORD=<tu_contraseña>

DB_SERVER=localhost

DB_DATABASE=dicri

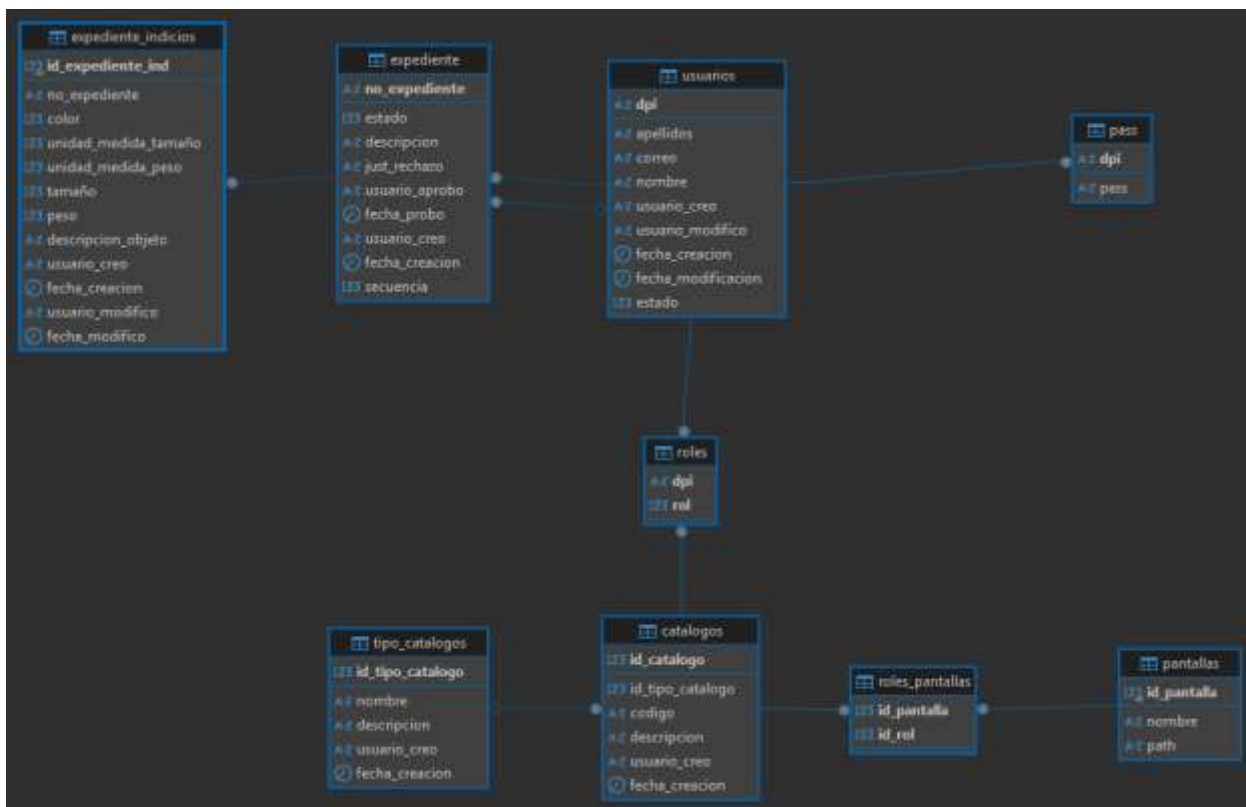
PORT=3000

JWT_SECRET=71135ece1a6f8658fd9f9ef482472062740993bd84ba3d06d74f

JWT_DURATION=8h

- Todas las operaciones de **lectura, creación, actualización y eliminación** se ejecutan mediante **procedimientos almacenados (EXEC)**.

- Los parámetros necesarios para cada procedimiento se envían desde Node.js utilizando **consultas parametrizadas**, garantizando seguridad frente a inyecciones SQL.
- Cada procedimiento encapsula la lógica de negocio y mantiene la integridad de la base de datos, incluyendo transacciones y manejo de errores (TRY/CATCH).
- La autenticación se gestiona mediante **JWT**, usando JWT_SECRET y JWT_DURATION para generar y validar tokens en cada solicitud.
- La aplicación mantiene un flujo modular mediante **rutas y controladores**, donde cada controlador llama a los procedimientos almacenados correspondientes para cumplir la operación deseada.



El modelo se divide en tres módulos lógicos:

1. **Módulo de Seguridad:** Gestión de identidades, credenciales y matriz de accesos
2. **Módulo de Catálogos (Core):** Centralización de datos maestros y listas desplegables.
3. **Módulo Operativo:** Gestión transaccional de expedientes y evidencias (indicios).

Módulo de Configuración (Catálogos)

Este módulo actúa como la fuente de verdad para todas las listas desplegables del sistema (Roles, Colores, Unidades de Medida, Estados).

Tablas:

- Tipos_catalogo
- catalogos
- roles_pantalla
- pantallas

Módulo de Seguridad y Control de Acceso

Este módulo es el encargado de la autenticación y autorización de usuarios dentro del sistema. Gestiona quién puede acceder (Login), qué perfil tiene asignado (Roles) y qué rutas del Frontend (React) tiene permitido visualizar.

Tablas:

- usuarios
- pass
- roles
- pantallas
- roles_pantallas

Detalle de las Tablas:

usuarios

- **Descripción:** Tabla maestra de empleados o personal con acceso al sistema.
- **Llave Primaria:** dpi (Documento Personal de Identificación). Se utiliza el identificador legal único de la persona como la llave primaria del sistema, evitando el uso de IDs autoincrementales para la identidad.

pass

- **Descripción:** Almacena las credenciales de acceso de los usuarios.
- **Seguridad:** Se mantiene en una tabla separada con una relación 1:1 respecto a la tabla usuarios. Esto se hace para aislar la información sensible (contraseñas) de los datos demográficos generales.
- **Campos Clave:** dpi (FK hacia usuarios), pass (hash de la contraseña).

roles

- **Descripción:** Tabla transaccional para la asignación de perfiles a usuarios.
- **Lógica de Negocio:** Vincula un dpi (Usuario) con un rol específico.
- **Integridad:** El campo rol es una llave foránea (FK) que apunta al id_catalogo en la tabla central de catalogos. Esto permite crear nuevos roles desde la configuración sin necesidad de alterar la estructura de la base de datos.

pantallas

- **Descripción:** Inventario de las rutas, vistas o componentes del Frontend (React).
- **Campos Clave:**
 - path: La ruta de navegación relativa (ej. /expedientes/nuevo).
 - nombre: La etiqueta visual que aparecerá en el menú.

roles_pantallas

- **Descripción:** Matriz de permisos (Tabla asociativa).
- **Función:** Determina qué roles (IDs provenientes de catalogos) tienen permiso explícito de visualizar qué pantallas.
- **Uso:** Al momento de iniciar sesión, el sistema consulta esta tabla para construir dinámicamente el menú de navegación del usuario según su rol.

Módulo Operativo (Negocio)

Este módulo gestiona la información transaccional central del sistema. Es el encargado de registrar el ciclo de vida de las carpetas de investigación (expedientes) y el inventario detallado de las evidencias físicas recolectadas, asegurando la trazabilidad de quién operó cada registro y estandarizando los atributos físicos mediante catálogos.

Tablas:

- expediente
- expediente_indicios

Detalle de las Tablas:

expediente

- **Descripción:** Representa la cabecera del caso o carpeta de investigación. Es la entidad principal del flujo de trabajo.
- **Flujo de Trabajo:** Controla el estado del caso mediante campos de aprobación y rechazo.
 - estado: Referencia al catálogo de estados (ej. Iniciado, En Revisión, Aprobado).
 - just_rechazo: Campo de texto para documentar las razones en caso de que un supervisor devuelva el expediente.
- **Auditoría de Usuarios:** Mantiene la integridad referencial hacia la tabla usuarios mediante:
 - usuario_creo: Quién inició el expediente.
 - usuario_aprobo: Quién validó el expediente (Supervisor).

expediente_indicios

- **Descripción:** Almacena el detalle de los objetos o evidencias físicas recolectadas asociadas a un expediente específico.
- **Cardinalidad:** Mantiene una relación de "uno a muchos" (1:N) con la tabla expediente.
- **Normalización de Datos:** Para garantizar reportes estadísticos limpios, los atributos físicos no almacenan texto libre, sino referencias a IDs:
 - color, unidad_medida_tamano y unidad_medida_peso son llaves foráneas (FK) que apuntan a la tabla catalogos.
 - Esto evita errores de digitación (ej. "rojo", "Rojo", "ROJO") y unifica la data.

VI. Repositorio

FRONTEND

git clone -b frontend <https://github.com/Juarezgump655/expedientes.git>

BACK

git clone -b back <https://github.com/Juarezgump655/expedientes.git>

DOCUMENTACIÓN

git clone -b doc-rama <https://github.com/Juarezgump655/expedientes.git>